



CICLO 1 - 2026

ANÁLISIS Y DISEÑO DE SOFTWARE

UNIVERSIDAD DE EL SALVADOR - FMOcc

Departamento de Ingeniería y Arquitectura

Docente: Licdo. Edwin González



Avisos y Anuncios - Semana 3

- Actividades a Desarrollar
 - Participación en el Foro
 - Leer y estudiar los contenidos de esta semana
 - Realización del examen corto #1



Conceptos Generales

Unidad 1

Fundamentos de análisis y Diseño
de Software



Agenda

1.3 Metodologías de Desarrollo de Software

1.3.1 Metodología Convencional

1.3.3 Metodología Orientada a Objetos

1.3.2 Metodología Estructurada

1.3.4 Metodología Ágil

1.3

Metodologías de Desarrollo de Software



Introducción



Una metodología de desarrollo de software se refiere a un enfoque sistemático, estructurado y coherente para planificar, diseñar, implementar, probar y mantener sistemas de software. Estas metodologías proporcionan un marco de trabajo que guía a los equipos de desarrollo a lo largo de todo el ciclo de vida del desarrollo de software, desde la concepción de la idea hasta la entrega del producto final.

Introducción



Una metodología de desarrollo de sistemas no tiene que ser necesariamente adecuada para usarla en todos los proyectos. Cada una de las metodologías disponibles es más adecuada para tipos específicos de proyectos, basados en consideraciones técnicas, organizacionales, de proyecto y de equipo.

Definición



Definición

Una metodología es un conjunto integrado de técnicas y métodos que permite abordar de forma homogénea y abierta cada una de las actividades del ciclo de vida de un proyecto. Es un proceso de software detallado y completo. Las metodologías se basan en una combinación de los modelos de proceso genéricos. Definen artefactos, roles y actividades, junto con prácticas y técnicas recomendadas. (Maida & Pacienza, 2015, p.12)

Metodologías de Desarrollo de Software

La diferencia fundamental entre cada metodología está en la forma en que se organiza el trabajo que se debe realizar. Algunas primero hacen un levantamiento general de la solicitud y luego dividen el trabajo en componentes más pequeños, otras hacen un prototipado evolutivo hasta llegar al resultado deseado.



Metodologías de Desarrollo de Software



Todas las metodologías buscan lo mismo: que los requerimientos del cliente, planteados funcionalmente, se cumplan a cabalidad y con un producto final fácil de usar, es decir ergonómico e “intuitivo” y eficiente al utilizar los recursos de procesamiento y de la red en general.

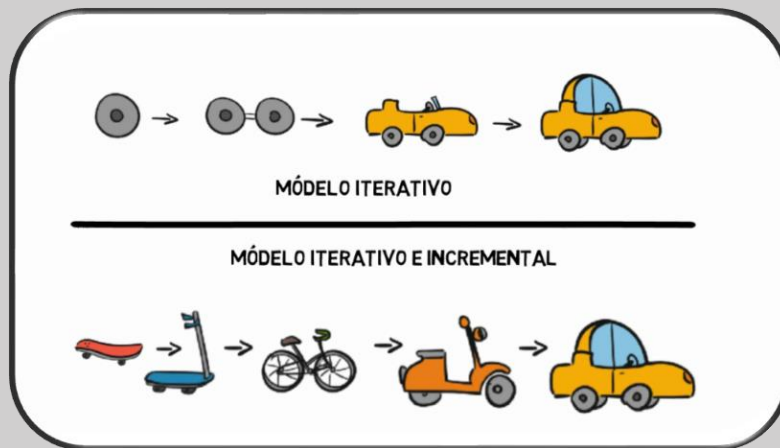
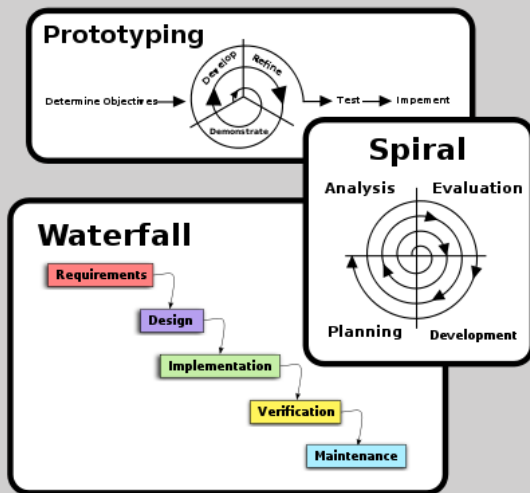
1.3.1 Metodologías Convencionales



1.3.1 Metodologías Convencionales

Las metodologías convencionales o tradicionales se rigen bajo una estructura secuencial inalterable basada en etapas. Dichas etapas son: el análisis de requerimientos, diseño, programación, pruebas y mantenimiento. Se le da gran importancia a los requerimientos del proyecto. Dado que se precisa de un amplio estudio para su definición, y una vez establecidos, no pueden alterarse de ninguna manera.

Ejemplos de Metodologías Convencionales





METODOLOGIA	CARACTERISTICAS	CASOS DE USO	VENTAJAS	DESVENTAJAS
Cascada	Secuencial, fases bien definidas.	Proyectos con requerimientos estables y bien definidos.	Fácil de entender y seguir. Documentación detallada.	No maneja bien los cambios. El cliente no ve el software hasta el final.
Espiral	Combina iteración con gestión de riesgos. Permite cambios.	Proyectos complejos y de alto riesgo.	Control preciso de costos y riesgos. Permite retroalimentación continua.	Puede ser costoso y llevar más tiempo debido a las iteraciones constantes.
Prototipo	Crea prototipos rápidos para obtener retroalimentación del cliente.	Proyectos donde los requerimientos no están completamente claros.	Facilita la presentación de resultados al cliente. Reduce malentendidos.	Puede ser costoso si se crean múltiples prototipos.
Incremental	Divide el proyecto en módulos funcionales que se entregan por separado.	Proyectos donde se necesita entregar funcionalidades rápidamente.	Permite cambios en los requerimientos en cada iteración.	Costos más elevados debido a la necesidad de integración de módulos.
RAD (Desarrollo Rápido)	Enfocado en la entrega rápida y la adaptación a cambios.	Proyectos donde se necesita rapidez y flexibilidad.	Prioriza las necesidades del cliente. Entrega rápida de versiones funcionales.	Requiere tiempos de entrega precisos y puede ser difícil de gestionar.
Modelo en V (V-Model)	Enfatiza la verificación y validación en cada fase.	Proyectos donde la calidad y la precisión son críticas (sistemas médicos, aeroespaciales).	Garantiza calidad y trazabilidad. Reduce errores desde el principio.	Rígido y poco flexible ante cambios. Complejidad en la gestión.
Basado en Componentes (CBD)	Reutiliza componentes preexistentes para acelerar el desarrollo.	Proyectos con elementos reutilizables (tiendas en línea, sistemas de autenticación).	Ahorro de tiempo y costos. Mayor calidad y escalabilidad.	Dependencia de componentes externos. Limitaciones en la personalización.



1.3.2 Metodología Estructurada



1.3.2 Metodología Estructurada

La metodología estructurada es un enfoque sistemático y organizado para abordar problemas complejos y llevar a cabo tareas o proyectos de manera ordenada y eficiente. En el contexto del desarrollo de software, la metodología estructurada se refiere a un conjunto de principios y técnicas que guían la planificación, diseño, implementación y evaluación de proyectos de desarrollo de software de manera disciplinada.

1.3.2 Metodología Estructurada

En el ámbito del desarrollo de software, una metodología estructurada implica la definición de pasos, reglas y normas que deben seguirse durante todo el ciclo de vida del desarrollo del software. Estos pasos se diseñan para asegurar que el proceso de desarrollo sea coherente, predecible y capaz de producir software de alta calidad.

Una metodología estructurada generalmente incluye:

División en Fases o Etapas:

El proceso se divide en fases bien definidas, como la recolección de requisitos, el diseño, la implementación, las pruebas y la entrega. Cada fase tiene objetivos específicos y produce resultados concretos.

Documentación Rigurosa:

Se enfatiza la creación de documentos detallados que describen los requisitos, el diseño, la arquitectura y otros aspectos del software. Esto garantiza la comprensión compartida entre los miembros del equipo y permite una comunicación más clara.

Enfoque en la Planificación:

Se da una gran importancia a la planificación y al establecimiento de cronogramas realistas. La planificación cuidadosa ayuda a evitar retrasos y a garantizar la finalización exitosa del proyecto.

Una metodología estructurada generalmente incluye:

Diseño Estructurado:

Se emplea un enfoque detallado para el diseño del software, donde los componentes se descomponen en partes más pequeñas y manejables. Esto facilita el desarrollo, la depuración y el mantenimiento.

Control de Calidad y Pruebas:

Se dedica tiempo y esfuerzo significativos a las pruebas para garantizar que el software funcione correctamente y cumpla con los requisitos especificados.

Gestión de Cambios:

Se establecen mecanismos para manejar los cambios en los requisitos o en el diseño a lo largo del proceso de desarrollo.

Seguimiento y Evaluación:

Se realiza un seguimiento constante del progreso del proyecto y se llevan a cabo evaluaciones regulares para garantizar que el software cumpla con los estándares de calidad y los objetivos del proyecto.

En resumen, una metodología estructurada en el desarrollo de software busca proporcionar un marco sólido para abordar la creación de software de manera metódica, ordenada y controlada.

1.3.3 Metodología Orientada a Objetos



1.3.3 Metodología Orientada a Objetos

La metodología Orientada a Objetos modela sistemas como un conjunto de objetos que interactúan entre sí. Utiliza pilares como: abstracción, encapsulación, herencia y polimorfismo, empleando principalmente UML(Unified Modeling Language) para diagramar la estructura y el comportamiento del sistema.

1.3.3 Metodología Orientada a Objetos

Principios fundamentales:

- Abstracción: Expresa las características esenciales de un objeto, las cuales distinguen al objeto de los demás.
- Encapsulación: Es la agrupación del estado y comportamiento para formar objetos, donde algunas partes son visibles, mientras que otras permanecen ocultas.
- Modularidad: Descompone el problema en pequeños bloques que realicen una función en específico.

1.3.3 Metodología Orientada a Objetos

Características principales:

- El sistema se modela como un conjunto de objetos que interactúan entre sí.
- Reduce la complejidad en el diseño de software.
- Se enfoca en identificar responsabilidades y relaciones entre objetos.

1.3.4 Metodología Ágil



1.3.4 Metodología Ágil

La metodología ágil es una metodología iterativa, es decir, se realizan entregas cíclicas y en cada entrega se realizan todas las fases del ciclo: desde toma de requerimientos, diseño, verificación y entrega. La mayor diferencia de las metodologías ágiles, frente a los antiguos modelos waterfall, es que en los procesos ágiles se entrega valor y recibe feedback constantemente durante todo el proyecto. En lugar de depender de un producto final predefinido desde el inicio, estas metodologías promueven la creación de software funcional que evoluciona de manera incremental.

1.3.4 Metodología Ágil

Las fases del proyecto, como el desarrollo, diseño y prueba, se realizan en ciclos cortos, conocidos como sprints, permitiendo que el equipo se ajuste rápidamente a los cambios y reciba feedback de los clientes de forma continua. Después de cada sprint, los equipos reflexionan y observan lo que ha sucedido. Evalúan si hay algo que se podría mejorar para poder ajustar la estrategia para el siguiente sprint.

1.3.4 Metodología Ágil

Las metodologías ágiles se basan en el “Manifiesto Ágil”, este establece cuatro valores principales:

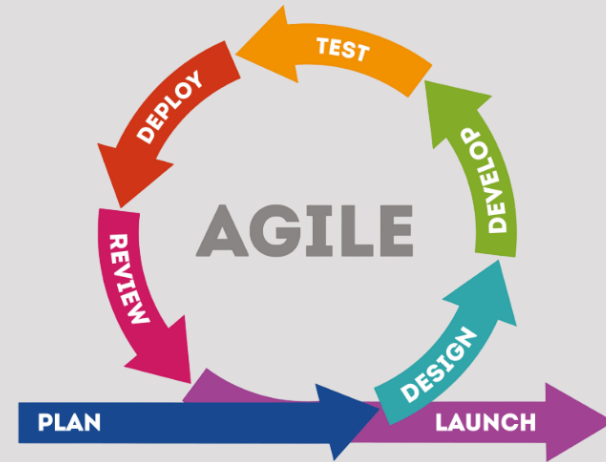
VALORES ÁGILES



1.3.4 Metodología Ágil

Ejemplos de metodologías ágiles:

- Scrum
- Kanban
- Extreme Programming (XP)
- Desarrollo de Software Lean
- Crystal
- Desarrollo Dirigido por Comportamiento (BDD)



Recordatorios

- Investiga más sobre los temas vistos en clase.
- Participa en foros.
- Realiza el examen corto #1.
- Si tienes dudas, ponte en contacto con tu tutor.



¡Gracias!



Icons made by storyset.com from @flaticon